

Air University



Cryptography (Lab)

Year 2026

Mr. Omer Ali

Academic Year: 2024 – 2028

Department of Cyber Security

USAMA ARSHAD | 247141

BS – Cyber Security – Fall – 24 (A)

Date of Submission: April 23, 2026

Lab # 10

1. Setup:

i. Create Working Directory:

```
(kali@kali)-[~]  
$ cd Documents  
  
(kali@kali)-[~/Documents]  
$ mkdir Lab_10  
  
(kali@kali)-[~/Documents]  
$ cd Lab_10
```

ii. Create a plaintext file:

```
(kali@kali)-[~/Documents/Lab_10]  
$ nano Lab_File.txt
```

iii. Contents of the file:

```
(kali@kali)-[~/Documents/Lab_10]  
$ cat Lab_File.txt  
Hi there, This is Lab 10.  
We are exploring the AES encryption using OpenSSL.  
This is the Secret message for encryption.
```

2. AES-256-CBC

i. Encryption:

```
(kali@kali)-[~/Documents/Lab_10]  
$ openssl enc -aes-256-cbc -salt -in Lab_File.txt -out Encrypted_File.txt -pbkdf2  
enter AES-256-CBC encryption password:  
Verifying - enter AES-256-CBC encryption password:
```

ii. Ciphertext:

```
(kali@kali)-[~/Documents/Lab_10]  
$ xxd Encrypted_File.txt  
00000000: 5361 6c74 6564 5f5f 1623 ae33 d8be 562a Salted_#.3..V*  
00000010: b28b e639 e8b9 fb63 0c04 0119 58ff b89f ... 9 ... c ... X ...  
00000020: 7d83 c534 053a 3ff2 9480 3a60 4857 e382 } ..4.?: ... :`HW..  
00000030: 9502 3bd6 9d24 7489 6684 932e 3348 2623 ..; ..$.f...3H6#  
00000040: da7f a738 9cb2 6ac5 fb26 c9e9 c250 c1c6 ... 8 .. j .. & ... P..  
00000050: 3b22 14ca b6a6 aa7a 5338 a4c6 2d97 4eda ;".....zS8 ..-.N.  
00000060: be5a 3879 496c 5dfd 1969 a32a 6464 ea2f .Z8yIL] ..i.*dd./  
00000070: 2ed4 e9a0 5c3b 7233 a525 9532 dfdc b662 . ... \;r3.%.2 ... b  
00000080: 8781 0dc0 ecee 46cd 8c89 6df1 9d9d e042 ... .. F ... m ... B  
  
(kali@kali)-[~/Documents/Lab_10]  
$ base64 Encrypted_File.txt  
U2FsdGVkX18WI64z2L5WKrKL5jnouftjDAQ8GVj/uJ99g8U0BTo/8pSA0mBIV+OCLQI71p0kdIIm  
hJMuM0gmI9p/pzicsmrF+ybJ6cJQwcY7IhTKtqaqelM4pMYtl07avlo4eUlsXf0ZaaMqZGTqLy7U  
6aBc03IzpSWVMt/ctmKHgQ3A705GzYyJbfGdneBC  
  
(kali@kali)-[~/Documents/Lab_10]  
$ cat Encrypted_File.txt  
Salted_#33V*+++9+++c  
+++F+m+++B
```

iii. Decryption:

```
(kali@kali)-[~/Documents/Lab_10]  
$ openssl enc -aes-256-cbc -d -in Encrypted_File.txt -out Decrypted_File.txt -pbkdf2  
enter AES-256-CBC decryption password:
```

iv. Verification:

```
(kali@kali)-[~/Documents/Lab_10]
$ cat Decrypted_File.txt
Hi there, This is Lab 10.
We are exploring the AES encryption using OpenSSL.
This is the Secret message for encryption.

(kali@kali)-[~/Documents/Lab_10]
$ diff -y Lab_File.txt Decrypted_File.txt
Hi there, This is Lab 10.
We are exploring the AES encryption using OpenSSL.
This is the Secret message for encryption.
Hi there, This is Lab 10.
We are exploring the AES encryption using OpenSSL.
This is the Secret message for encryption.
```

3. AES-256-ECB

i. Encryption:

```
(kali@kali)-[~/Documents/Lab_10]
$ openssl enc -aes-256-ecb -in Lab_File.txt -out ECB_Encrypted_File.txt -pbkdf2
enter AES-256-ECB encryption password:
Verifying - enter AES-256-ECB encryption password:
```

ii. Ciphertext:

```
(kali@kali)-[~/Documents/Lab_10]
$ xxd ECB_Encrypted_File.txt
00000000: 5361 6c74 6564 5f5f c885 7c3c 07e1 918b  Salted__..|<....
00000010: 40f3 a357 dffe ad93 80b4 06f2 8a10 3a08  @..W.....:
00000020: 9a52 5184 cc46 6276 3b07 7d5e 4019 884e  .RQ..Fbv;.^@..N
00000030: 3660 c9e2 319b 66fe da9d 2e6a 7381 0e6c  6`..1.f...js..l
00000040: 9454 d4d1 0c4b f0fb 6690 7057 df52 4990  .T...K..f.pW.RI.
00000050: 63ab 1be4 3442 3cee 3f2b 7146 3400 fcd5  c...4B<.+qF4...
00000060: 20c7 b5d4 15de 4f38 5512 10cc 10d4 a6b7  ....08U.....
00000070: a0d7 912c 5ea7 a671 0a5a ff0b 0a56 c20b  ...^..q.Z...V..
00000080: 4dfa 6ff6 9717 2aaa 8096 3664 8024 16b0  M.o...*...6d.$..

(kali@kali)-[~/Documents/Lab_10]
$ base64 ECB_Encrypted_File.txt
U2FsdGVkX1/IhXw8B+GRi0Dzo1ff/q2TgLQG8ooQ0giaUlGEzEZidjsHfV5AGYhONmDJ4jGbZv7a
nS5qc4EObJRu1NEMS/D7ZpBwV99SSZBjqxvkNEI87j8rcUY0APzVIMe11BXeTzhVEhDMENSmt6DX
kSxep6ZxClr/CwpWwgtN+m/2lxcqoCWNmSAJBaw

(kali@kali)-[~/Documents/Lab_10]
$ cat ECB_Encrypted_File.txt
Salted__@W+++++++RQ+Fbv;.^@N6`+1+f. js+L+T++
K+f+pW+RI+c+4B<+?+qF4+ g+08U+h+.,^++q
Z+
V+
M+o+++++6d+$+
```

iii. Decryption:

```
(kali@kali)-[~/Documents/Lab_10]
$ openssl enc -aes-256-ecb -d -in ECB_Encrypted_File.txt -out ECB_Decrypted_File.txt -pbkdf2
enter AES-256-ECB decryption password:
```

iv. Verification:

```
(kali@kali)-[~/Documents/Lab_10]
$ cat ECB_Decrypted_File.txt
Hi there, This is Lab 10.
We are exploring the AES encryption using OpenSSL.
This is the Secret message for encryption.

(kali@kali)-[~/Documents/Lab_10]
$ diff -y Lab_File.txt ECB_Decrypted_File.txt
Hi there, This is Lab 10.
We are exploring the AES encryption using OpenSSL.
This is the Secret message for encryption.
Hi there, This is Lab 10.
We are exploring the AES encryption using OpenSSL.
This is the Secret message for encryption.
```

4. Short Report:

Differences Between ECB and CBC Modes

The two most common AES modes produce fundamentally different security properties despite using the same underlying cipher. The table below summarises their key differences based on observed OpenSSL output:

Aspect	ECB Mode	CBC Mode
Block Processing	Each block is encrypted independently — no state is carried between blocks	Each block is XOR-ed with the previous ciphertext block before encryption
IV Required	No IV needed; every block is self-contained	16-byte random IV required to seed the first block XOR
Pattern Security	Identical plaintext blocks → identical ciphertext (patterns visible)	Identical plaintext blocks → different ciphertext (patterns hidden)
Error Propagation	A corrupt ciphertext block affects only that one block on decryption	Corrupt block garbles its own decrypted block and the immediately following block
Use Case	Unsafe for most real data; acceptable only for single-block random data	Recommended for all multi-block encryption; standard in TLS, disk encryption
OpenSSL flag	-aes-256-ecb (no -iv parameter needed)	-aes-256-cbc (OpenSSL auto-generates IV when -salt is used)
output	"HELLO" encrypted twice with the same key → same ciphertext both times	"HELLO" encrypted twice with the same key but different IVs → different ciphertext each time

In practice, running the same ECB encryption command twice with the same password produces an identical ciphertext file, while running the CBC command twice produces a different ciphertext each time because OpenSSL generates a fresh random salt and IV on every invocation. This directly demonstrates why CBC is the standard choice for any real-world encryption.

Output Comparison:

Feature	Encrypted_File.txt	ECB_Encrypted_File.txt
Magic Bytes (Hex)	53 61 6c 74 65 64 5f 5f	53 61 6c 74 65 64 5f 5f
ASCII Header	Salted__	Salted__
Salt Value (Hex)	16 23 ae 33 d8 be 56 2a	c8 85 7c 3c 07 e1 91 8b
Base64 Start	U2FsdGVkX18WI64z2L5W...	U2FsdGVkX1/IhXw8+GRi...
Cipher Mode Context	"Likely a standard block mode (e.g., CBC)."	Explicitly labeled as ECB (Electronic Codebook) in the filename.
File Structure	Consists of 9 lines of hex data (approx. 144 bytes).	Consists of 9 lines of hex data (approx. 144 bytes).
Visual Patterns	High entropy; no immediate visual repetition in the hex.	"While ECB often shows patterns, the ciphertext here still appears randomized (likely due to high-entropy plaintext)."

5. Open SSL Commands:

The table below explains each OpenSSL command used in this lab in 1–2 lines:

OpenSSL Command	Explanation
openssl enc -aes-256-cbc -salt -in Lab_File.txt -out Encrypted_File.txt -pbkdf2	Encrypts Lab_File.txt using AES-256-CBC with a random salt and PBKDF2 key derivation, writing the binary ciphertext to Encrypted_File.txt.
openssl enc -aes-256-cbc -d -in Encrypted_File.txt -out Decrypted_File.txt -pbkdf2	Decrypts Encrypted_File.txt using AES-256-CBC with the same password and PBKDF2 derivation, recovering the original plaintext into Decrypted_File.txt.
openssl enc -aes-256-ecb -in Lab_File.txt -out ECB_Encrypted_File.txt -pbkdf2	Encrypts Lab_File.txt using AES-256-ECB (no IV) with PBKDF2 key derivation, writing the ciphertext to ECB_Encrypted_File.txt.
openssl enc -aes-256-ecb -d -in ECB_Encrypted_File.txt -out ECB_Decrypted_File.txt -pbkdf2	Decrypts ECB_Encrypted_File.txt using AES-256-ECB and the same password with PBKDF2 derivation, writing the recovered plaintext to ECB_Decrypted_File.txt.

Flag Reference

- **enc** — Invokes the OpenSSL symmetric cipher encoding/decoding sub-command.
- **-aes-256-cbc / -aes-256-ecb** — Specifies the cipher algorithm and mode (CBC or ECB) with a 256-bit key.
- **-salt** — Prepends an 8-byte random salt to the output; combined with PBKDF2, it prevents identical passwords from producing identical keys across runs.
- **-d** — Switches the operation from encryption to decryption.
- **-in <file>** — Specifies the input file path.
- **-out <file>** — Specifies the output file path for the resulting ciphertext or plaintext.
- **-pbkdf2** — Uses PBKDF2 (Password-Based Key Derivation Function 2) with HMAC-SHA256 to derive the AES key and IV from the password, replacing the deprecated -md md5 default. Required by OpenSSL 3.x to suppress deprecation warnings.

- End of Lab Task -